DEVELOPMENT CHRONICLE & ROADMAP

Student Codex

The complete build record — every phase, fix, and refinement from the first scaffold to the release candidate — together with the roadmap of upcoming modifications and innovations.

StudentCodex Project · Partido State University

OCTOBER 24, 2026 · RECORD NO. 1

Table of Contents

1. About This Chronicle 1

2. How StudentCodex Was Built 1

3. Development History 2

3.1 Foundation (Phase 0-1) . . . 2

3.2 Visual precision (two passes) . . . 3

3.3 The database (Phase 2) . . . 3

3.4 Backend integration . . . 3

3.5 Making it functional (Phase 4) . . . 4

3.6 The floating assistant and the great restructure (P5) . . . 4

3.7 Hardening and the release candidate . . . 4

4. The Consolidated Edit Log 5

5. Engineering Lessons Learned 7

5.1 Working across the bridge . . . 7

5.2 Android and Compose . . . 7

5.3 Supabase and PostgREST . . . 7

5.4 Process . . . 8

6. Roadmap: Upcoming Modifications and Innovations 8

6.1 Near term: already designed and staged . . . 8

6.2 Mid term: building on the foundation . . . 9

6.3 Long term: the innovation arc . . . 9

7. Closing Note 10

1. About This Chronicle

This document is the construction record of StudentCodex: a complete, honest account of every tracked work package from the first scaffold to the current release candidate, together with the roadmap of what comes next. It is written alongside the Project Abstract, which describes the system as it stands; this volume describes how it got that way, one verified edit at a time.

The record is drawn directly from the project's engineering worklog, a journal kept by the build agents throughout development. Each entry in that log carries a task identifier, the work performed, and a stage summary that states what was verified before the task was closed. Nothing in this chronicle is reconstructed from memory; every milestone below traces back to a logged, test-backed change on the developer's real machine.

The chronicle serves three purposes. For the developer, it is the authoritative history of the codebase - what changed, in what order, and why. For future contributors, it is institutional memory that prevents re-learning hard-won lessons. And for the project itself, it is the baseline against which the roadmap in Chapter 6 will be executed and tracked: every future task will enter this same log with the same verification bar.

29	117	11	0
tracked work packages in the build log	automated unit tests in the release candidate	database migrations applied to the live backend	lint errors across the whole project

2. How StudentCodex Was Built

StudentCodex was built through an unusual but rigorous collaboration model: an AI engineering agent operating inside a cloud sandbox, driving the developer's real Windows machine through a bridge - file reads, writes, and edits plus command execution against the actual VS Code workspace where the project lives. Every line of code described in this chronicle was written to that real workspace, compiled with the machine's own Gradle and JDK 21 toolchain, and installed on the machine's Android emulator. There was never a throwaway prototype branch; the record and the product are the same artifact.

Verification was the non-negotiable center of the process. A change was not accepted until it passed the full chain: compile, the complete unit-test suite, lint with zero errors, an assembled APK, installation on the emulator, and - for anything user-facing - on-device verification through interface dumps and screenshots examined by a vision model. The project's own success criteria stated it plainly: no feature is called complete when only the UI exists.

Database work followed a database-first safety rule. Every migration was written and tested against a sandboxed real PostgreSQL 18.4 instance with a Supabase platform mock - roles, JWT-claim functions, and storage schema simulated - before it was ever pushed to the hosted project. Row-level-security policies were exercised as executable test scenarios, both the allowed and the denied paths, because a policy that has never been proven to reject an unauthorized request is only a hope.

Finally, the process ran on documentation as a first-class artifact: a product requirements document as the approved baseline, architecture notes and decision records for every major choice, and the running worklog itself. The method mattered because the project moved fast - the distance from empty directory to release candidate was covered inside a single intensive window in late October 2026, with the verification chain as the only brake on speed.

3. Development History

The history divides naturally into seven movements: foundation, visual precision, the database, backend integration, making everything functional, the floating-assistant and navigation restructure, and the polish passes that produced the release candidate. This chapter tells the story in order; Chapter 4 tabulates every individual work package.

3.1 Foundation (Phase 0-1)

Development began with the plan itself: a 723-line build plan revision that fixed the architecture as Supabase-first - Supabase Auth, PostgreSQL with row-level security, Storage, and Edge Functions, with no intermediate server of our own. Phase 0 audited the build configuration (Android Gradle Plugin 9.2.1, Kotlin 2.2.10, compileSdk raised to 37 for the newest androidx artifacts) and wrote the requirements, architecture, and decision documents. Phase 1 delivered the first 25 files in one pass: the design-token system, the application scaffold with five tabs and navigation, domain models, repository interfaces with deliberately fake demo implementations, the deterministic guidance engine, and the first unit tests. The remote build then went green: fourteen tests passing, zero lint errors, and the first debug APK assembled on the developer’s machine.

3.2 Visual precision (two passes)

Before any backend work, the interface was put through two dedicated precision passes against the workspace's interface-quality standards. The first pass replaced every default: a library-green-on-warm-paper palette, a disciplined corner-radius system, content-shaped skeleton loaders instead of spinners, flat dense list rows with dividers, and silhouette author marks instead of initial-letter avatars. The second pass delivered the deferred refinements: the DM Sans variable typeface as a bundled asset, a hand-tuned surface elevation ramp, the edge-to-edge padding fix that had left headers under the status bar, and interactive suggestion chips in the guidance chat. The visual identity established here held for the rest of the project.

3.3 The database (Phase 2)

The schema arrived as six versioned migrations: profiles, roles, and the academic registry; courses, sections, and enrollments; activities, sources, reports, and participants; the community tables; moderation; and storage policies. Each policy shipped with a test, and the test loop earned its keep by catching four real production bugs before they could ship - a SELECT-policy evaluation trap on INSERT RETURNING, an UPDATE visibility rule that made soft-deletes impossible for authors, a policy that recursed infinitely on its own table, and a trusted-context bypass that guard triggers had accidentally blocked. Twenty-four policy scenarios, roughly ninety assertions, all green on real PostgreSQL before anything touched the hosted project.

3.4 Backend integration

Supabase entered the app through a fail-fast configuration pipeline: a git-ignored environment file validated at Gradle time - missing files, malformed keys, and any hint of service-role material abort the build - feeding exactly two BuildConfig fields. The client library was pinned to supabase-kt 3.6.0 after an empirical version probe showed newer releases require a newer Kotlin metadata than the toolchain provides. Byte-level scans of every compiled dex confirmed that no privileged key material ships in the APK. After the developer's one-time CLI setup, the six migrations were pushed to the hosted project, and the schema was confirmed live through the same REST channel the app uses - with row-level security verified by anonymous requests returning zero rows.

3.5 Making it functional (Phase 4)

The request that reshaped the product was simple - make it functional, make every button work - and it produced the authentication gate: real email and password sign-up and sign-in with sessions that survive restarts, a guest bypass for evaluation, and a wiring audit that left no dead control in the app. Google Sign-In followed, using the Android Credential Manager with a Google ID token exchanged by Supabase Auth, and with it a debugging saga worth recording: the generic failure message was replaced with per-cause diagnostics, which then walked the real rollout - no Google account on the device, Play services rejection, the disabled-provider error - until the flow worked end to end. Sign-up matured into its final shape in stages: first and last names collected with a derived username, a completion form on first Google sign-in that also sets a password, server-side blocking of duplicate accounts, and school-details selection from database-backed registry dropdowns rather than hardcoded lists.

3.6 The floating assistant and the great restructure (P5)

The guidance page then dissolved into the rest of the app. First it became a floating assistant: a Messenger-style draggable chat head that docks to the screen edge, plus an alternative edge-panel style, switchable and persisted. The guidance tab left the bottom bar shortly after - the assistant had replaced it. A Settings page was born to hold the style switch along with real password and school-details editing, and the chat head was restyled to the developer’s reference and then polished on feedback: the tail wedge removed, the edge tuck set to seventy percent peeking. The restructure completed when Settings became the bottom tab, Profile became a pushed page, the community gained a working composer with real photo attachments, and every path to guidance converged on one chat card and one transcript. The teacher dashboard - a role-gated Next.js view on the same backend with section workload, deadline preview, and report triage - landed in the same window.

3.7 Hardening and the release candidate

The closing passes were the ones that made it a product. A live PGRST200 error on Home was traced to two broken PostgREST embed queries and fixed with the alias-target form, verified against the hosted database before rebuilding. The Beacon was simplified to a single write path. Sign-up was reordered campus-first with the official ParSU program registry - forty-two program-campus rows - so students see only what their campus actually offers. The edge handle learned to rest half-tucked. And the Settings page evolved through five successive refinement tasks into its final form: an operations page of action cards opening modal bottom sheets, with sign-out after every operation, no category headers, the Google Classroom connection card and the notifications row migrated in from Profile, and Profile itself reduced to a pure identity page. Every one of these passes ended the same way: build green, tests green, install, and on-device verification.

4. The Consolidated Edit Log

The table below records every tracked work package in the engineering worklog, in order, from the bridge migration that made remote development possible to the final Profile-to-Settings migration. References in the first column are the task identifiers under which each package was logged and verified.

Ref	Work package	Delivered and verified
1	Operator bridge migration	Sandbox-to-VS-Code bridge stood up, dashboard browser-verified, services hardened to survive session cleanup.
2	Phase 0-1: foundation	Build plan adopted; Gradle config audited; design tokens, five-screen scaffold, fake repositories, demo guidance engine; 25 files written.
3	Phase 1 verification	Remote compile fixed; compileSdk raised to 37; 14 unit tests green; lint zero errors; first debug APK.
4	Phase 2: database	Six versioned migrations; 24 row-level-security policy scenarios on real PostgreSQL 18.4; four production bugs caught pre-emptively.
UI-1	Interface precision, pass 1	Library-green-on-warm-paper re-theme; content-shaped skeletons; flat dense rows; silhouette author marks.
UI-2	Interface precision, pass 2	DM Sans variable typeface bundled; surface elevation ramp; edge-to-edge status-bar fix; interactive guidance suggestion chips.
5	Supabase integration	Fail-fast .env-to-BuildConfig pipeline; supabase-kt 3.6.0 pinned after version probe; dex scans clean; Google Classroom source-identity test added.
S-11	Integration completion	Real-config build chain fully green; Supabase CLI enabled on the developer machine with working link-and-push instructions.
S-12	Hosted go-live	Migrations applied to the hosted project; schema and RLS verified live through the same REST channel the app uses.
P4-A	Auth gate and wiring	Email/password sign-up and sign-in before the app; guest bypass; every previously dead control wired; 30 unit tests.
P4-B	Google Sign-In	Credential Manager flow with Supabase token exchange; honest setup dialog until the OAuth client is configured.
P4-C	Community comments	Comment counts became real buttons opening a thread sheet with a working reply box; 12 new tests.
P4-D	Google sign-in bug	Per-cause error diagnostics replacing the generic failure; OAuth client and SHA-1 registration documented.

Ref	Work package	Delivered and verified
P4-E	Provider diagnosis	Root-caused the disabled Google provider in Supabase; Android side proven working end to end.
P4-F	Names and duplicates	First and last names at sign-up with derived usernames; Google completion form sets a password; duplicates blocked server-side.
P4-G	School-details sign-up	Program, year level, section, and campus chosen from database-backed registry dropdowns in both sign-up forms.
P5-A	Floating assistant	Guidance became a draggable chat head plus an optional edge panel, switchable and persisted, over every page.
P5-B	Navigation cleanup	Guidance removed from the bottom bar; the floating assistant replaced the tab.
P5-C	Settings page created	Assistant style, password change, and school-details editing moved into a new Settings page; chat head restyled with ring, shadow, and unread badge.
P5-D	Chat head polish	Tail wedge removed; edge tuck changed to seventy percent visible per feedback.
P5-E	Navigation restructure	Settings became the tab and Profile a pushed page; functional community composer with photo attachments; one shared guidance surface.
11-a	Teacher dashboard	Role-gated web dashboard live: section filters, workload summaries, proposed-deadline preview, and the report-triage chain.
P5-F	PGRST200 hotfix	Broken PostgREST embeds rewritten in alias-target form and verified against the live database; Home, Beacon, and Profile loading real data.
P5-G	Beacon simplification	Report-activity button removed in favor of the single Add-activity path; effort input dropped.
P5-H	Campus-first sign-up	Official ParSU registry seeded - 42 program-campus rows; campus chosen first, programs filtered to that campus in sign-up and Settings.
P5-I	Half-tucked edge handle	Edge-panel button rests fifty percent off-screen; tapping it opens the same guidance card as the chat head.
P5-J	Settings operations	Change-password and change-school-details buttons open modal bottom sheets; sign-out moved after all operations.
P5-K	Settings header removal	Category headers removed; Settings reads as one flat list of cards.
P5-L	Profile-to-Settings migration	Google Classroom connection card and notifications row moved to Settings; Profile reduced to a pure identity page.

Table 1. The consolidated edit log: every tracked work package, in order.

5. Engineering Lessons Learned

A project built this way accumulates lessons that are worth writing down, because they are the difference between the next feature taking an hour or a day. The most durable ones follow, grouped by where they were learned.

5.1 Working across the bridge

- Windows source files use CRLF line endings; edits that ignore this fail silently or match nothing. Large insertions are delivered as fragment files appended by script, never as interactive diffs.
- Shell commands on the remote machine are capped at 200 seconds; Gradle builds run detached with log polling instead of in the foreground.
- Never invoke the git execution alias on the remote machine - the Windows Store stub can block the persistent shell session entirely.

5.2 Android and Compose

- Experimental Material 3 APIs (modal bottom sheets among them) require explicit opt-in annotations; resolving all of them at once beats whack-a-mole compiles.
- Any edge-docked draggable must exclude system gestures; without it, an inward drag triggers system navigation and exits the app.
- Suspend calls cannot run inside non-suspend state-update lambdas; hoist them into the coroutine before the update.
- Under the current toolchain, serialization codegen is silently absent - runtime JSON parsing is the reliable path.

5.3 Supabase and PostgREST

- Embedded queries must use the alias-target form; the bare form fails with a schema-cache error on the live service.
- An upsert against a partial unique index must repeat the index predicate in the ON CONFLICT clause, or it fails to find the constraint.
- Row-level-security denials are silent zero-row no-ops - tests must assert row state, not exceptions - and UPDATE enforces SELECT policies as WITH CHECK.

5.4 Process

- Verify, then ship: the compile-test-lint-install-verify chain caught every regression before it reached the developer.
- Honest interface states are a feature: loading skeletons, empty states, and per-cause error messages turned bug reports into diagnoses.
- Unique artifact names and exact-node parsing made on-device verification reliable; substring checks and size heuristics did not.

6. Roadmap: Upcoming Modifications and Innovations

The roadmap is organized by readiness, because this project's future work is unusually well staged: the nearest items already have their schema, interfaces, or designs in place, and each was deferred deliberately rather than forgotten. Every item below will enter the worklog under a task identifier and meet the same verification bar as everything in Chapter 4.

6.1 Near term: already designed and staged

- Live Google Classroom synchronization. An Edge Function performs the OAuth code exchange, keeps refresh tokens in its private store, calls the existing service-role import RPC per coursework item, and writes connection status to the registry. The unique source key, the idempotent RPC, the connection card, and the coverage card already exist; no app or schema changes are required.
- Notifications and reminders. The Settings row exists; the work is push delivery, deadline reminders, and the preference screen behind it.
- Offline cache and outbox. Implement the existing design: a Room cache for reads, an outbox for writes, and WorkManager-driven retry-safe synchronization, so the Beacon and Community work without network and reconcile when it returns.

6.2 Mid term: building on the foundation

- Community polish: three photos per post from the composer (the schema already permits it), a richer moderation queue for reviewers, and an author edit window.
- A smarter assistant: an LLM-backed engine behind the same chat surface - the transcript, badge plumbing, and fallback contract are ready - grounded in app context and held to the same honesty rules.
- Workload analytics: scoring and week-over-week insight for students, calendar and widget views, and richer teacher analytics on the dashboard.
- Guidance depth: study tools such as summaries and spaced-repetition review built from the student's own activity list.

6.3 Long term: the innovation arc

- Predictive workload: suggesting better publication dates to teachers before deadlines cluster, and personal load forecasts for students.
- Cross-section assignment groups for team work spanning sections, on the participant model that already exists.
- Multi-school expansion: the registry is database-backed by design, so a second institution is a data operation, not a code change.
- Public release: signed release builds, a store listing, staged rollout, and an accessibility and localization pass (Filipino first).

Horizon	Modification or innovation	Foundation already in place
Near	Google Classroom live sync	Source keys, import RPC, connection registry, UI cards
Near	Notifications and reminders	Settings preference row; Supabase Auth user records
Near	Offline cache and outbox	Complete design document: Room, outbox, WorkManager
Mid	Smarter guidance assistant	One chat surface; badge plumbing; honest fallback contract
Mid	Community polish	Schema permits three images; moderation queue exists
Mid	Workload analytics and widgets	Workload views and due-soon counts already computed
Long	Predictive workload suggestions	Section workload aggregates on the teacher dashboard

Horizon	Modification or innovation	Foundation already in place
Long	Multi-school expansion	Registry is database-backed, not hardcoded
Long	Public store release	Release-candidate build discipline already in place

Table 2. Roadmap summary with the foundations each item builds on.

7. Closing Note

Twenty-nine tracked work packages took StudentCodex from an empty directory to a release candidate: a real backend with tested row-level security, an Android app whose every control does something honest, a teacher dashboard that closes the loop between the two audiences, and a staged path to the one integration that remains. The through-line of the whole record is the verification chain - nothing shipped on faith, and nothing was called done that had not been watched working on a real device.

The chronicle does not end here. The next entries in this log are already named: the Classroom Edge Function, notifications, the offline cache, and the long arc of analytics and expansion beyond them. The discipline that produced this record is the project’s most important artifact, and it is the one that carries forward unchanged.